Créez votre première page en Symfony

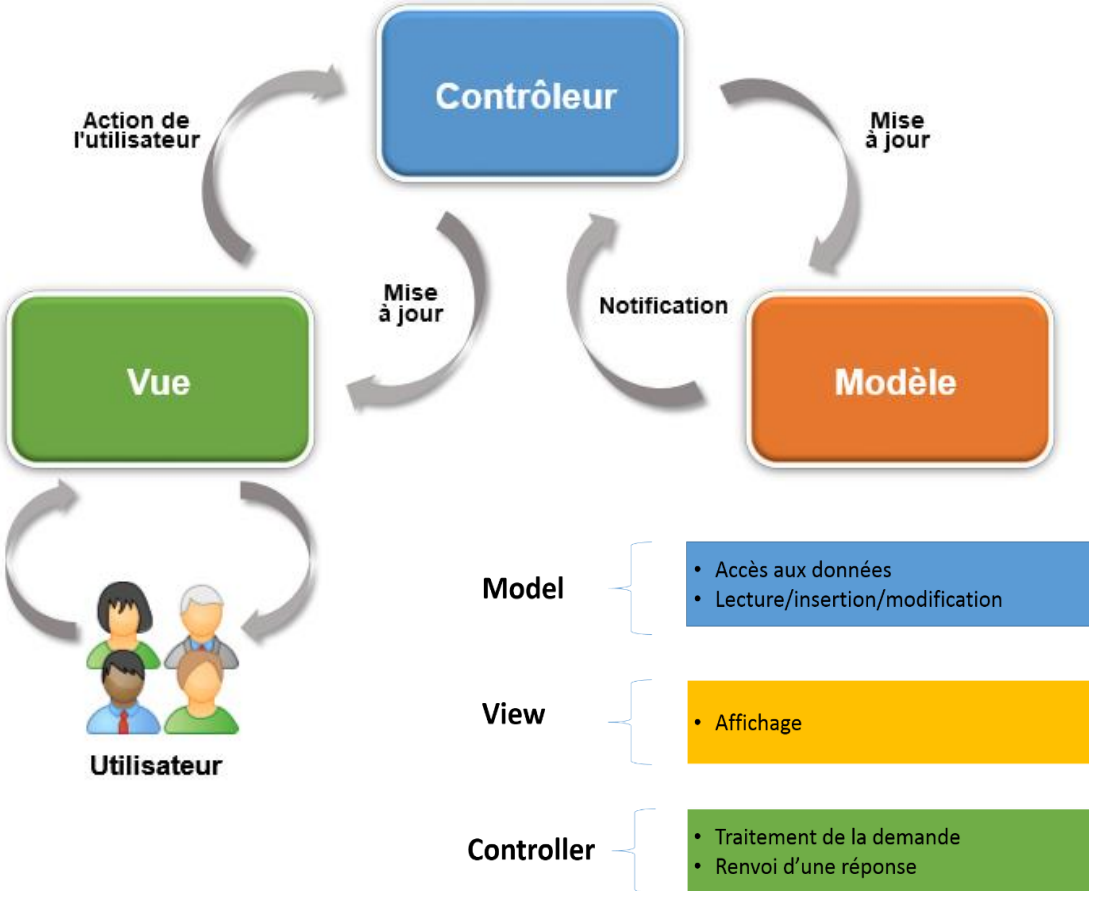
I. Le MVC

Le design pattern MVC utilisé par Symfony consiste à diviser notre application en 3 couches :

Modèle (Model) : Cette section gère les données, les règles d'affaires et la logique. Pensez-y comme la base de données ou l'endroit où toutes les informations sont stockées et gérées.

Vue (View) : C'est ce que l'utilisateur voit et interagit avec. Il s'agit de l'interface utilisateur, comme les pages web, qui présente les données au utilisateur.

Contrôleur (Controller) : Cette partie agit comme un intermédiaire entre le Modèle et la Vue. Il gère la logique de traitement des requêtes de l'utilisateur, puis interagit avec le Modèle pour récupérer des données ou effectuer des changements, et met à jour la Vue en conséquence.



Voici un exemple simple pour illustrer comment le modèle MVC (Modèle-Vue-Contrôleur) fonctionne dans le cadre d'une application web développée avec Symfony :

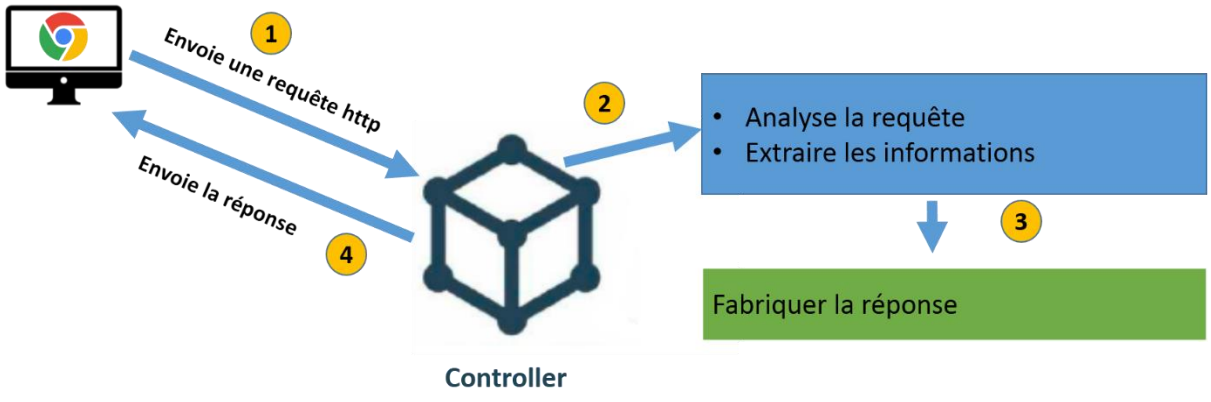
Modèle (Model) : Imaginons que vous avez une application web pour gérer une bibliothèque. Le modèle est responsable de gérer les données de la bibliothèque, comme les informations sur les livres (titre, auteur, année de publication, etc.) et les membres (noms, adresses, historique d'emprunt, etc.). Le modèle interagit avec la base de données pour sauvegarder et récupérer ces informations.

Vue (View) : La vue est la partie de l'application que les utilisateurs voient et avec laquelle ils interagissent. Par exemple, il peut y avoir une page web qui affiche une liste de livres disponibles dans la bibliothèque. Cette page est générée par la vue, qui récupère les informations nécessaires du modèle et les affiche dans un format convivial pour l'utilisateur.
Contrôleur (Controller) : Le contrôleur agit comme un intermédiaire entre le modèle et la vue. Disons qu'un utilisateur veut emprunter un livre. Il sélectionne un livre sur la page web et soumet une demande. Cette demande est envoyée au contrôleur, qui la traite. Le contrôleur demande ensuite au modèle de mettre à jour les informations (par exemple, marquer le livre comme emprunté). Après la mise à jour, le contrôleur demande à la vue d'afficher une confirmation ou de mettre à jour la liste des livres disponibles.
Dans cet exemple, chaque partie du MVC a un rôle clair :
Le **Modèle** gère les données et la logique métier.
La **Vue** affiche les informations aux utilisateurs et recueille leurs interactions.
Le **Contrôleur** fait le lien entre les utilisateurs (via la vue) et les données (via le modèle).
Cette séparation des responsabilités rend l'application plus organisée, facile à gérer et à étendre.

On va s'attarder dans cet atelier sur le concept de **contrôleur**, vu qu'il représente le cœur d'une application symfony.

1. Le concept du Controller

Le rôle du Controller va être de nous retourner une réponse par rapport à une route ou une requête donnée.



II. Arborescence des fichiers d'une application symfony

En examinant attentivement le contenu du projet SymApp. Les dossiers et sous-dossiers sont organisés comme suit :

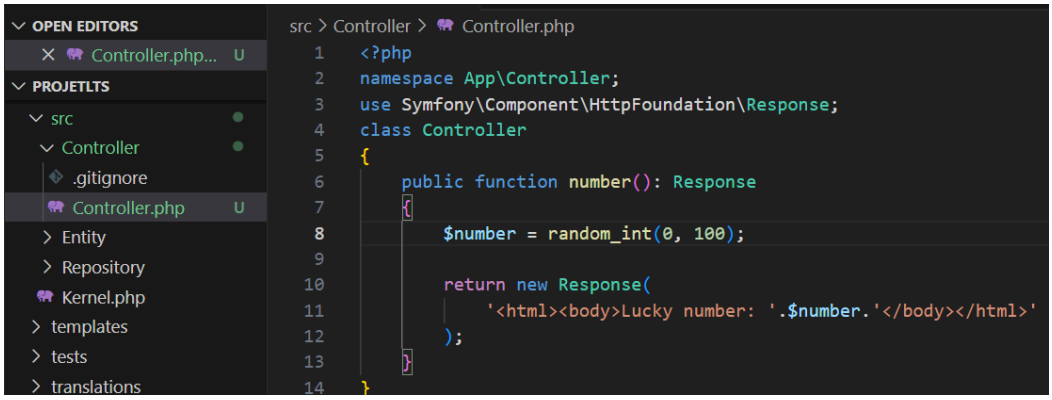
/**bin** : Ce répertoire contient des exécutables, principalement le script console de Symfony, qui est utilisé pour exécuter des commandes Symfony et des tâches d'administration.
/**config** : Ce dossier contient tous les fichiers de configuration de votre application. Cela inclut les configurations de routes, services, packages, et tout autre paramétrage spécifique à votre application.
/migrations : Ce dossier est crucial pour gérer la base de données dans une application Symfony. Il contient des fichiers de migration, qui sont des scripts PHP utilisés pour modifier la structure de la base de données de manière contrôlée et versionnée. Ces fichiers définissent des opérations telles que la création ou la modification de tables, l'ajout d'index, etc. Les migrations permettent de suivre les changements apportés à la structure de la base de données au fil du temps et de les appliquer de manière cohérente dans différents environnements de développement, de test et de production.
/**assets** : Ce répertoire est utilisé pour stocker les ressources statiques de l'application, telles que les fichiers JavaScript, les feuilles de style CSS, et les images. Ces ressources sont essentielles pour la partie front-end de l'application web.
JavaScript : Ici, vous pouvez placer vos scripts JavaScript, qu'ils soient personnalisés ou des bibliothèques tierces.
CSS : De même, les fichiers CSS pour le style de votre application sont stockés dans ce dossier.
Images : Toutes les images utilisées dans votre application peuvent être stockées ici.
/**public** : C'est le document root de votre application web. Il contient le fichier index.php, qui est le point d'entrée de votre application. Les fichiers statiques comme les images, les fichiers JavaScript et CSS sont également servis à partir de ce répertoire.
/**src** : Ce dossier est le cœur de votre application. Il contient votre code source PHP, y compris :
Contrôleur (Controller) : Les classes de contrôleurs gèrent la logique d'interaction entre le modèle et la vue.
Entity (Modèle) : Les classes d'entité représentent les structures de données de votre application et sont généralement mappées à la base de données.
Repository : Ces classes fournissent des fonctions pour interroger et récupérer des données de la base de données.
Form : Classes pour gérer les formulaires.
Services : Classes qui contiennent la logique métier de l'application.
/**templates** : Contient les fichiers de template Twig, qui sont utilisés pour générer les vues de votre application.
/**tests** : Contient les tests unitaires et fonctionnels de votre application.
/**translations** : Ce répertoire contient les fichiers de traduction pour internationaliser votre application.
/**var** : Ce dossier contient des fichiers générés par Symfony, tels que les fichiers de cache et les journaux (logs).
/**vendor** : Les bibliothèques tierces installées via Composer sont placées dans ce répertoire.
.env : Fichier pour configurer les variables d'environnement de l'application.

composer.json et composer.lock : Ces fichiers sont utilisés par Composer pour gérer les dépendances PHP de votre projet.

La création d'une nouvelle page est un processus en deux étapes :

1. Créer un contrôleur : Un contrôleur est la fonction PHP que vous écrivez pour construire la page. Vous utilisez les informations de la requête entrante pour créer un objet Symfony Response, qui peut contenir du contenu HTML.
2. Créer une route : Une route est l'URL (par exemple, /contact) de votre page et pointe vers un contrôleur.

Supposons que nous souhaitons créer une page - /number - qui génère un nombre chanceux (aléatoire) et l'affiché. Pour ce faire, créez une classe "Controller" et une fonction "number" à l'intérieur de celle-ci :




```php
<?php
namespace App\Controller;
use Symfony\Component\HttpFoundation\Response;
class Controller
{
    public function number(): Response
    {
        $number = random_int(0, 100);

        return new Response(
            '<html><body>Lucky number: '.$number.'</body></html>'
        );
    }
}
```

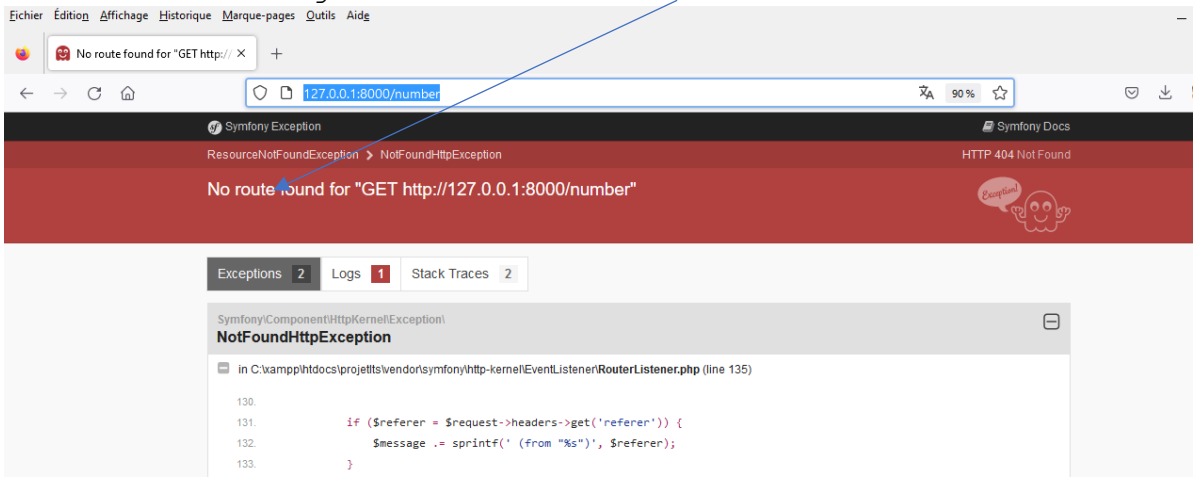
```
namespace App\Controller;    //c est le chemin complet du controller, App c'est la racine du projet, Controller est le nom du répertoire Controller où il y a la classe Controller.php. On n'a pas indiqué le répertoire src car symfony sait que Controller se trouve dans src. Car en réalité ça doit être App\src\Controller
```

```
use Symfony\Component\HttpFoundation\Response;
```

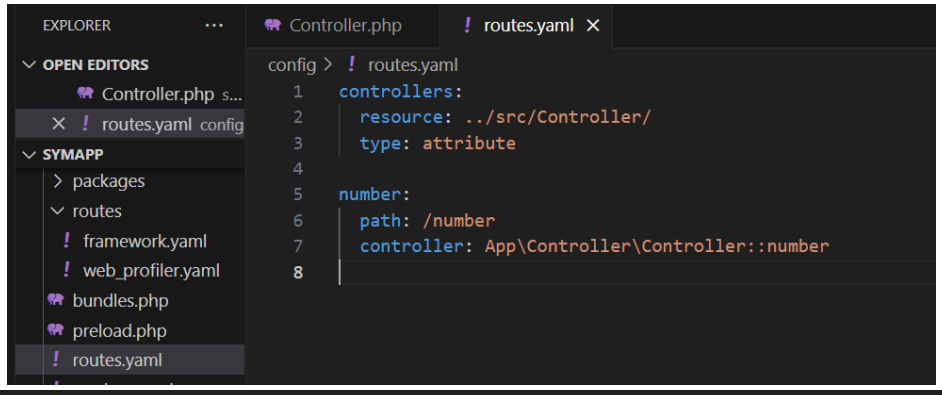
La classe Response est essentielle pour gérer les réponses HTTP dans une application Symfony.

Response (): Cette partie indique que la méthode retourne une instance de la classe Response. En d'autres termes, après l'exécution de cette méthode, elle doit retourner un objet Response, qui est couramment utilisé pour renvoyer une réponse http, par exemple, retourner un numéro, une page ou des données au format JSON...

le nom de la fonction est important, car s'il est diffèrent de index on doit l'indiquer dans l'url. Dans notre cas on doit taper http://127.0.0.1:8000/number
On aura un message d'erreur « No route found »



Maintenant, nous devons associer cette fonction de contrôleur à une URL publique (par exemple, /number) de manière à ce que la méthode number() soit appelée lorsque l'utilisateur y accède. C'est la deuxième étape citée au-dessus.
On va aller sur le fichier config\routes.yaml et le modifier




```yaml
number:
  path: /number
  controller: App\Controller\Controller::number
```

number: Il s'agit d'une configuration pour une route nommée " number ". Dans Symfony, une route nommée permet de donner un nom spécifique à une route, ce qui

peut être utile pour la référencer ou la lier à d'autres parties de l'application. On peut mettre n’importe quel nom.

path: /number: Cela définit le chemin de l'URL auquel cette route sera associée. Cela signifie que lorsque les utilisateurs accèdent à l'URL "/number" dans leur navigateur, cette route sera activée.

controller: App\Controller\Controller::number: Cela spécifie le contrôleur qui sera responsable de gérer les requêtes à l'URL "/number". Plus précisément, cela indique que la méthode "number" du contrôleur "App\Controller\Controller" doit être exécutée lorsque quelqu'un accède à l'URL "/number". Cette méthode sera chargée de générer la réponse appropriée pour cette requête.